PortfolioOptimisers.jl

Daniel Celis Garza[1]

[1]Independent researcher, Oxford, UK

October 20, 2026

Abstract

Portfolio optimisation is the science of either: *1)* Minimising risk whilst keeping returns to acceptable levels. *2)* Maximising returns whilst keeping risk to acceptable levels. To some definition of acceptable, and with any number of additional constraints available to the optimisation type. There exist myriad statistical, pre- and post-processing, optimisations, and constraints that allow one to explore an extensive landscape of "optimal" portfolios. PortfolioOptimisers.jl is an attempt at providing as many of these as possible under a single banner and making it accessible to all: 57 risk measures, 22 covariance estimators, 13 prior estimators, 20 optimisers and 22 online portfolio selection rules, which compose with each other rather than sit in separate silos, and which a single cross-validation framework can tune as one object. The library reads a point-in-time universe, in which assets list, delist and are suspended, and its estimators can be updated one observation at a time rather than refitted. We make extensive use of Julia's type system, module extensions, and multiple dispatch to simplify development and maintenance, while keeping robustness, testability, and usability high.

Keywords. Portfolio Optimisation · Quantitative Investment · Conic Optimisation · Parameter Estimation

1 Introduction

The field of portfolio optimisation was introduced by Harry Markowitz's seminal 1952 paper, *Portfolio Selection* [1]. The field has grown enormously since then, and so has the catalogue of that model's failure modes. It attempts to summarise the entire distribution of returns in highly compressed summary statistics, both of which can be sensitive to outliers and atypical conditions for the lookback period. The optimiser then behaves as an error maximiser rather than a diversifier [2], is acutely sensitive to the expected returns it is given [3], and suffers roughly an order of magnitude more from errors in the means than from errors in the covariance [4]. The equal weighted portfolio remains a stubborn out of sample benchmark [5]. Sixty years of responses, from constraints [6] and shrinkage [7] to norm regularisation [8], robust formulations and hierarchical ones, are by now a literature of their own [9], [10], [11], [12].

Unfortunately, most implementations live in disparate, unconnected, often proprietary, and/or bespoke codebases, which limit their applicability. It is only recently that various advanced yet very usable libraries have been published [13], [14], [15]. However, each has its own strengths, weaknesses, scope, and idiosyncrasies. That is no different from PortfolioOptimisers.jl, but it is our hope that this library serves as a unifying framework for the various techniques and methods available in the field, while also eventually providing a simple and intuitive interface for users, with advanced features for experts.

There also exist myriad optimisation methods, pre-filtering, distribution and moment estimators, machine learning techniques, validation, and parameter tuning methods that can all be used together to improve the out-of sample performance of a portfolio. To this day, only skfolio [15] has succeeded in providing a unified framework for this. PortfolioOptimisers.jl provides an alternative that is not tied to the scikit-learn [16] or cvxpy [17], [18] APIs and ecosystems, as well as providing different functionality and a different architectural philosophy.

Table 1 gives a sense of the scale. What follows is a tour rather than a manual: the documentation carries the detail, and the point of this paper is to show that the detail is there.

2 Design and implementation

2.1 Basic example

PortfolioOptimisers.jl is built with modularity and extensibility in mind. We can demonstrate a simple improvement over the Markowitz model by adding an L2 regularisation term to the optimisation problem, which shrinks the weights towards the equal weighted portfolio and is known to improve out of sample performance [8].

$$\begin{aligned} \min_{\boldsymbol{w}} \quad & \boldsymbol{w}^\top \boldsymbol{\Sigma} \boldsymbol{w} + \lambda \left\| \boldsymbol{w} \right\|_2^2 \\ \text{s.t.} \quad & \boldsymbol{w}^\top \boldsymbol{\mu} \geq \mu_i \\ & \boldsymbol{w}^\top \mathbf{1} = 1 \\ & 0 \leq w_j \leq 1 \quad \forall j \neq \text{AAPL} \\ & 0 \leq w_{\text{AAPL}} \leq 0.2 \end{aligned} \tag{2.2}$$

The problem is solved once per return level μ_i, for $N = 100$ levels evenly spaced between the minimum and maximum attainable returns, which traces the efficient frontier of Figure 2.

Table 1: Concrete estimator and algorithm types per family, counted from the library's own type tree; the Result types they return are not counted. The 20 optimisers are 6 naïve, 5 JuMP.jl based, 4 clustering, 2 ensemble, 2 finite allocators, and 1 hindsight comparator. One of the naïve optimisers holds any of the 22 online portfolio selection rules, and 24 named constructors fix a rule to its published form. Composition multiplies them: any covariance estimator feeds any distance, any distance feeds any clustering or network algorithm, and any of those feeds any optimiser that asks for one.

Family	**Types**	**A few of them**
Risk measures	57	Variance, CVaR, EVaR, RLVaR, their range and drawdown twins, ordered weights, high order moments
Covariance and correlation	22	Gerber, Smyth-Broby, Kendall, Spearman, mutual information, distance covariance, lower tail dependence, geodesic shrinkage, regime adjusted exponential weighting
Expected returns	12	Equilibrium, excess, median, windowed, exponentially weighted, price level forecasts, and shrunk towards three targets
Prior statistics	13	Empirical, time series and cross-sectional factor, high order, four Black-Litterman variants, entropy pooling, opinion pooling
Cross-sectional factor model	27	Descriptor estimators behind 37 named descriptors, such as book to price or momentum, factor exposures including observed and currency factors, and return forecasts
Phylogeny algorithms	14	Three minimum spanning trees, DBHT, hierarchical clustering, k-means, eight centrality measures
Uncertainty sets	5	Normal, ARCH bootstrapped, delta, characteristic, and orthogonal to a factor model, in box, ellipsoidal, norm ball or compact form
Calibration rules	14	Rules that compute a significance level, a deformation parameter, an ambiguity radius, a norm ceiling, or a factor mean blend from the data in place of a number
Constraints, costs, and penalties	18	Weight bounds, three budget forms, linear equations, thresholds, risk budgets, factor exposure, two phylogeny forms, centrality, fees, turnover, two allocation sets of an online rule, and L2 or Lp weight penalties. Custom constraints and objectives are an abstract type to subtype, so they are not counted
Optimisers	20	Equal weighted, inverse volatility, online portfolio selection, best constant rebalanced portfolio, mean risk, near optimal centering, risk budgeting, HRP, HERC, Schur, nested clustering, stacking, discrete allocation
Online portfolio selection rules	22	Buy and hold, constant rebalancing, mirror descent, Newton step, passive aggressive and confidence weighted mean reversion, pattern matching, expert mixtures
Cross-validation and tuning	8	K-fold, combinatorial, walk-forward by index or by date, hindsight split, multiple randomised, grid and randomised search. Every sequential scheme can also step online
Preprocessing and pipeline	9	Price ingestion, prices to returns, missing data filter, price gap fill, three asset selectors, train and test split, pipelines that bundle end-to-end workflows and can be cross-validated and tuned as a single unit

```
# Import packages.
using PortfolioOptimisers, CSV, TimeSeries, Clarabel, StatsPlots, GraphRecipes
```

```
# Load prices and turn them into returns.
X = TimeArray(CSV.File(joinpath(@__DIR__, "../../examples/SP500.csv.gz"));
              timestamp = :Date)
rd = prices_to_returns(X)
rd_train, rd_test = train_test_split(rd; test_size = 0.2)
# Mean risk optimisation, which minimises risk by default.
mr = MR(;
  # Variance, written directly as a quadratic expression.
  r = Variance(; alg = QuadRiskExpr()),
  opt = JuMPOpt(;
    # Solvers are tried in order until one of them succeeds.
    slv = Solver(; name = :clarabel, solver = Clarabel.Optimizer,
                 settings = Dict("verbose" => false)),
    # An estimator, so the bounds are built from the data: every asset
    # is capped at 1 and floored at 0, except AAPL, capped at 0.2.
    wb = WBE(; ub = "AAPL" => 0.2),
    # Maps asset names to their columns, and names to sets of assets.
    # Constraint estimators and some priors are built against this.
    sets = UniverseSets(; dict = Dict("nx" => rd.nx)),
    # Squared L2 penalty on the weights, lambda = 1e-4.
    l2 = L2Reg(; val = 0.0001, alg = QuadRiskExpr()),
    # Sweep 100 return levels: one solve each, one efficient frontier.
    ret = ArithmeticReturn(;
            settings = JuMPReturnsSettings(; lb = Frontier(; N = 100)))
  ) # opt
) # mr
# Fit on the training set, then score both sets.
res = optimise(mr, rd_train)
pred_train = predict(res, rd_train)
pred_test = predict(res, rd_test)
# Scenario based standard deviation, as a second order cone expression.
r = SCM(; alg = SOCRiskExpr())
plt = plot_measures(pred_train; x = r, label = "Training", zcolor = nothing)
plt = plot_measures(pred_test; x = r, plt = plt, label = "Test", zcolor = nothing,
                    markercolor = :red, ylabel = "Mean Return",
                    xlabel = "Standard Deviation")
savefig(plt, joinpath(@__DIR__, "fig1.svg"))
nothing
```

Figure 1: End-to-end example of a regularised Markowitz model. The listing is not a transcript: it is executed by the build, so it cannot drift away from the library.

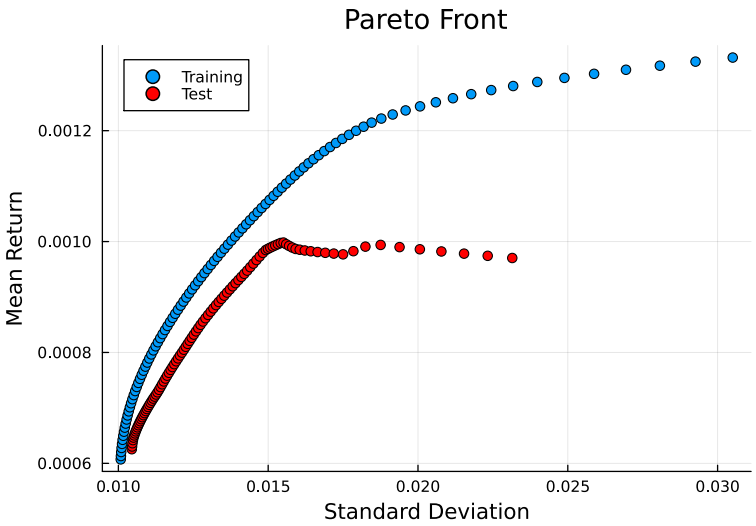


Figure 2: Train and test efficient frontier for an L2 regularised Markowitz model with a scenario-based standard deviation risk measure. The training set does much better than the test set, this is why other portfolio optimisation modalities have been invented.

2.2 Data flow

The library is configured to allow for different workflows. There is no rigid structure to the data flow. It is possible to use the library as simply as shown above or to build extremely complex workflows either manually, or using a pipeline. The functionality is grouped into three main stages, which break down into a hierarchy of standalone processes that can be used in isolation or as part of a larger workflow.

1. Preprocessing: data cleanup, filtering, and transformation.
   a. Ingestion: joins the asset, factor and benchmark tables, collapses the frequency, and records the listing span of every asset. It runs once over the whole history, because the folds of a cross-validation are cut on the dates it produces.
   b. Gap treatment: fills a missing price with something tractable, or drops the assets and observations with too many gaps.
   c. Preselection: uses various non-optimisation methods to preselect assets. Can operate at the price or returns stage.
   d. Returns computation: computes returns, and records which assets are listed and which can be estimated at every observation.

2. Processing: these can be provided precomputed, which is valid for certain optimisers and standalone optimisations, or computed inside an optimisation, which is required for certain meta optimisers and for cross-validation. The only requirement to perform an optimisation is a prior, and for certain optimisers, a clustering.
   a. Prior statistics: standalone moment estimation, integrative returns distribution estimation, and adjustments to the moment estimations.
   b. Phylogeny: uncovers the relational structure of the asset universe through clustering and network analysis.
   c. Constraint generation: lets users define constraints, some of which consume prior and phylogeny statistics.
   d. Uncertainty sets: used for robust optimisation, they ameliorate the sensitivity to estimation errors.
   e. Optimisation: consumes or internally carries out all the aforementioned steps and produces a result carrying every computed quantity.
3. Postprocessing: performance reporting, attribution, forecast evaluation, regret, and visualisation.

The preprocessing and processing steps can be bundled into a single pipeline object, which is itself an estimator. That pipeline can be cross-validated and tuned as a unit, so the preprocessing and processing stacks are tuned together rather than one at a time.

2.3 Point-in-time data

Real universes change. Assets list late, delist, and are suspended, and a backtest that drops them from the whole sample, or holds them after they are gone, suffers from survivorship and look-ahead bias. PortfolioOptimisers.jl never deletes a gap. The position of a gap in a price column says whether the asset was in the universe: a leading run means it had not yet listed, a trailing run that it delisted, and an interior run that it was suspended. The returns then carry an asset panel whose two masks say which assets are listed at each observation, and which of those have a finite return.

The estimators downstream read those masks. A moment estimator fits on the assets it can estimate over its window, and an optional coverage policy replaces that rule with available-case estimation, in which each entry of a covariance is fitted on the observations its two assets share. Every optimiser fits on the assets it can trade and gives every other asset a weight of exactly zero, so each fold of a walk-forward runs on the universe of that fold. The fees follow

the same calendar: proportional and turnover fees are charged on every observation, fixed fees once per holding period, and a liquidation charge on a position whose asset leaves the universe.

The same panel holds the features of the assets, such as a market capitalisation, a sector, or a graded classification, as named numeric, categorical or tensor fields. Feature distances and the cross-sectional factor model below read them from there. Each reader of the returns, such as a prior, a clustering, or an online rule, states whether it fits on the returns themselves, on their excess over a benchmark, or on the returns of the prior.

2.4 Moments

PortfolioOptimisers.jl comes with a large number of moment estimation and prior statistics methods. It leverages StatsBase.jl's covariance estimator definitions and API to allow interoperability with the Julia ecosystem, for example by composing with CovarianceEstimation.jl. They can be combined in many ways: it is possible to compute the expected returns using a specific covariance method and vice-versa.

- Full or downside co-moments.
- Weighted, unweighted.
- Shrinkage [7], and geodesic shrinkage, which moves along the affine-invariant geodesic between the sample estimate and a target, and so stays positive definite at every intensity [19].
- Exponentially weighted, and regime adjusted with an exact correction of the bias of the regime statistic.
- Probability-theoretic, including the Gerber statistic and its descendants [20], [21], [22].
- Denoising and detoning against the Marchenko-Pastur spectrum [11], [23].
- J-sparse, through information filtering networks [24].
- Regression-based.
- Rank based.
- Coskewness and cokurtosis.
- Expected returns forecast from the price levels, such as moving averages, spatial medians, and trend switches, as the online portfolio selection literature uses them [25].

2.5 Distance and phylogeny statistics

These bundle clustering and network analysis pipelines to explore the relational structure of the asset universe, and to derive insights that constraints and optimisers can consume. Clustering and network analysis are different, but they tackle similar issues from complementary angles,

and the same analysis techniques exploit the structure that each uncovers. As such we choose to categorise them under the same umbrella. The distance matrices include the first and second order distances defined in [11], and are compatible with every single covariance estimator that follows the StatsBase.jl API. The clustering and network analysis methods include:

- Distance, distance-of-distance and similarity matrices.
- Feature distances, which measure the assets by their exposures, memberships, factor loadings or graph neighbourhoods rather than by their returns.
- Hierarchical agglomerative clustering, with an exact optimal leaf order [26].
- Direct bubble hierarchy trees [27], [28].
- K-means clustering.
- Minimum spanning trees.
- Planar maximally filtered graphs [29].
- Centrality and community detection.

2.6 Prior statistics

A prior is the returns distribution the optimisation actually sees, and it is the one component every optimiser requires. They fall under three categories:

- Frequentist: empirical, timeseries factor, and cross-sectional factor models, for low and high moments.
- Bayesian: Black-Litterman [30], in vanilla, factor, Bayesian and augmented forms.
- Information-theoretic: entropy pooling [31], including views on CVaR and EVaR [32], and linear or logarithmic opinion pooling [33].

2.7 Cross-sectional factor models

The cross-sectional factor prior is the fundamental counterpart of the timeseries factor model [34]. It reads no series of factor returns. Instead, it builds the exposure of every asset to every factor from the asset panel, and at each observation it regresses the returns of the assets on their lagged exposures, so the factor returns are estimated rather than given. The model is assembled from parts that can be used on their own:

- Descriptors: values per asset and observation, computed from the panel fields, such as a book to price ratio, a momentum, or a downside beta. 37 are named, and each is a configuration of one of 17 general estimators.
- Exposures: composites of standardised descriptors, one-hot memberships such as sectors, and observed factors whose returns are read rather than estimated, such as a currency. Each

family of one-hot factors is collinear with a global factor, so the fit drops one member per family and expands the factor returns back afterwards.

- Return forecasts: predictions of the idiosyncratic return of each asset, fitted on the descriptor scores, whose spanned part is blended with the factor mean by precision.
- Diagnostics: the correlation, stability, dispersion and coverage of the exposures, the fit of each regression, and the standardised idiosyncratic returns. The first two groups are gathered into a single summary table, one row per factor.

The prior steps online. Each new observation is folded into the carried state of every part that has an exact update, and every other part is refitted over its rows, so a step reproduces the batch fit to round-off.

2.8 Risk measures

A risk measure is an object, not a hard-coded branch. Any optimiser that takes one takes a vector of them, scalarised into a single expression by a sum, a maximum, a minimum, or a log-sum-exp. Every prior-derived quantity a measure needs, such as the expected returns, the covariance, the coskewness or the cokurtosis, may be given as a value or as the estimator that computes it. In the second case it is refitted against the optimisation's own prior, once per cross-validation fold and once per subset of a meta-optimiser, which a pasted matrix cannot do.

- Dispersion: variance, standard deviation, low order moments, range, Brownian distance variance, and a variance taken over an uncertainty set.
- Tail: value at risk, conditional value at risk [35], entropic value at risk [36], power norm and relativistic value at risk [10], and worst realisation. Most have a range twin, and the conditional ones have distributionally robust variants [37].
- Drawdown: the same tail family applied to the drawdown series [38], plus average drawdown, maximum drawdown, and the ulcer index.
- Ordered weights: Gini mean difference, tail Gini, and L-moments of arbitrary order, as disciplined convex programmes [39], [40], [41], including an exact formulation through a sorting network whose number of constraints grows as $T \log^2 T$ in the number of observations [42].
- High order: coskewness and cokurtosis, full or semi, standardised or not.
- Relative: tracking error, risk tracking error and turnover, which measure deviation from a reference portfolio rather than from zero.

Twelve further measures are exclusive to the hierarchical optimisers, which can use measures that are not convex in the weights because they never place them in a solver.

The scalar parameters need not be numbers either. The tail probability or deformation parameter of a measure, the ambiguity radius of a measure or of a weight penalty, and a ceiling on the norm of the weights may each be a rule that the optimisation evaluates on its own prior, so it is recalibrated in every fold. The rules compute a significance level from the length of the sample, a deformation parameter from an estimate of the tail index, an ambiguity radius from the sampling error of the empirical measure [37], and a norm ceiling from a floor on the effective number of assets.

2.9 Uncertainty sets

Uncertainty sets are used for robust optimisation, where the parameters are only known to lie within a set. They ameliorate the sensitivity to estimation errors, and they currently cover the expected returns and the covariance.

- Box: the simplest set, defined by a lower and upper bound for each parameter [43].
- Ellipsoidal: defined by a radius and a covariance matrix, forming an ellipsoid in the parameter space [44].
- Characteristic: applies an L1 uncertainty set to the expected returns, which recovers the quintile portfolio as a special case [45]. This will eventually be generalised to any characteristic vector.
- Norm ball: the image of a norm ball under a map of any rank, so a set that is flat in the directions a factor model does not span needs no full rank shape matrix.
- Compact: holds the worst-case variance of a covariance set as a quadratic penalty on the weights, so the problem stays a second order cone programme with no semidefinite constraint.

The bounds themselves come from a normal approximation, from a delta method, or from an ARCH bootstrap of the estimator. A further estimator reads no returns at all: it takes the factor model of the optimisation's prior, and limits both sets to the directions that the factors do not span.

2.10 Optimisers

PortfolioOptimisers.jl provides a large number of optimisers, in six families.

1. Naïve: equal weighted, inverse volatility, random weighted, and previous weights, which holds the weights of the last fold. Speed and robustness, but not necessarily optimality. Interesting

as sub-optimisers to more complex optimisers, as fallbacks, and as the benchmark that is hardest to beat out of sample [5]. The family also holds the best constant rebalanced portfolio in hindsight, computed from Cover's fixed point without a solver [46].

2. Online portfolio selection: rules that move the weights after every new price relative, most of them without a solver [25]. A batch fit makes one causal pass over its window, and an online fit continues from the state the last one reached, so both give one path. The rules cover:
   a. Benchmarks: buy and hold, and constant rebalancing.
   b. Follow the winner: mirror descent in a Euclidean, entropic, Tsallis or log-barrier geometry [47], the Newton step [48], and the universal portfolio [46], with learning-rate schedules, optimistic gradient predictions, and any risk measure as the loss.
   c. Follow the loser: passive aggressive [49], moving average [50], robust median [51] and confidence weighted [52] mean reversion, anticorrelation [53], and transaction cost optimisation [54].
   d. Pattern matching: solves for the leader over the past rows that resemble the present ones [55], through a solver free fixed point or any JuMP.jl based optimiser.
   e. Meta-learning: mixes any of the above as experts, including mixtures built for changing environments [56].

   A prediction charges the fees of the head, with each trade measured against the held book. The optimiser keeps the weights of every rule inside an allocation set, either weight bounds that project in closed form or the constraints of a JuMP.jl based optimiser. A missing return is read as cash, so a rule never trades on a return that did not happen.
3. JuMP.jl based [57]: the most flexible and powerful, but they require a solver, such as Clarabel [58], and are typically slower than first-order optimisers. They support a wide range of constraints and objectives, and they use conic reformulations throughout. The family covers mean risk, near optimal centering, risk budgeting [59] and its relaxed form, and factor risk contribution.
4. Hierarchical: these use the relational structure of the universe to compute the risk of each group and sub-group, producing a diversified portfolio that encodes the relational structure as well as the risk characteristics of each group. They are typically faster than JuMP.jl based optimisers and support fewer constraints, but they are very good for large universes. The family covers hierarchical risk parity [60], hierarchical equal risk contribution [61], and Schur complementary allocation [62], which interpolates between hierarchical risk parity and minimum variance.

5. Meta-optimisers: these consume other optimisers and combine their results. They are typically slower, but can be very effective. The family covers nested clustering [11], stacking [63], and subset resampling [64].
6. Finite allocators: these do not optimise portfolios in the traditional sense. The user provides a finite cash amount and the asset prices, and the allocator computes the best portfolio attainable with that cash. They run after the others have produced a result. A long-short book either spends the proceeds of its short sales, or caps its gross exposure at a collateral amount.

2.11 Constraints, objectives, and penalties

There is a huge variety of constraints. Every optimiser supports weight bounds, but most of the richness is exclusive to JuMP.jl based estimators. Constraints are built by estimators rather than written as matrices, so a constraint is stated once and rebuilt against whatever data it meets. Linear constraints can be written as equations in near-natural language, for instance `"AAPL <= 0.1"`, `"MSFT >= AMD"`, or `"tech >= 0.15"` for a set of assets declared in the universe sets.

1. Weight bounds and budgets: the maximum and minimum weights, and the total value of the weights. Together they express leveraged, dollar-neutral, and long-short portfolios, and they integrate with the finite allocators, which adjust the available cash to the budget.
2. Linear constraints: used for relative weights, group exposures, risk contributions, and relational structure.
3. Cardinality constraints: sparsity, buy-in thresholds, and inclusion or exclusion rules, at the level of assets or of sets of assets. They are implemented as mixed integer linear constraints.
4. Fees and turnover: long and short proportional fees, fixed fees, and turnover fees, which penalise the returns and therefore, indirectly, the risk measures and the objective. The fixed fees are charged once per holding period, or amortised over it.
5. Weight penalties: L-norm penalties on the objective, or hard limits on the norm of the weights, which regulate sparsity and robustness.
6. Tracking: limits on how far a portfolio may drift from a reference, in weights or in risk.
7. Objectives: minimum risk, maximum return, maximum risk-adjusted return, and maximum utility, where the risk term may itself be a scalarised vector of risk measures.
8. Risk upper and return lower bounds: applicable simultaneously, and bounded by a scalar, a precomputed vector or range, or a frontier object that lets the optimiser compute the bounds on the fly. Bounding more than one quantity by a range produces a grid, and therefore a Pareto surface returned as a vector of results.

9. Custom constraints and objectives: user-defined terms that receive the optimiser's own context, and enter the objective as penalty contributions.
10. Time dependent: every constraint and every non-finite optimiser can be wrapped so that it varies across cross-validation folds. The wrapper takes a predefined list, or a function of the fold's context and the previous portfolio's weights, which allows a strategy to change its own constraints as it walks forward.

2.12 Incremental estimation

Some statistics have an exact update, in which the new value follows from a running state and one new observation. The sample mean, the variance, the full covariance, the coskewness and the cokurtosis are among them, and their estimators fold each new observation into that state rather than read the sample again. Every other estimator keeps the observations it has seen, optionally capped to a rolling window, and runs its batch computation over them. Both forms answer the same call, and every optimiser updates through its prior, so a whole strategy can step forward one observation at a time.

2.13 Cross-validation, hyperparameter tuning, and pipelines

The library comes equipped with a pipeline framework that can run an entire strategy end to end, and with a cross-validation framework that evaluates the out of sample performance of an optimisation or of a whole pipeline. The schemes include k-fold, index and date based walk-forward with purging, multiple randomised splits, and combinatorial splits that produce many backtest paths rather than one [65]. The same machinery drives the nested clustering and stacking meta-optimisers, which need out of sample predictions from their inner estimators before the outer optimisation can run.

Every sequential scheme also has an online form, in which each fold updates one estimator with its new observations instead of refitting it, and a finished run can be resumed over a longer history. A hindsight split trains each fold on the row it tests, which yields the per-row comparators of dynamic regret. A fold can also be scored on the weights a fund actually holds, which drift with the returns of their assets between trades, and those held weights can be passed on to the next fold.

The hyperparameter tuning framework is built on top of the cross-validation framework. Users address the steps of a pipeline by index or by name, and the fields to tune by name. Grid search takes grids of values, and random search takes grids, distributions from Distributions.jl [66], or both.

2.14 Evaluation and attribution

The postprocessing stage scores what the processing stage produced. The forecast evaluations also accept bare arrays, so they score a forecast from outside the library in the same way as one the library made.

- Performance: annualised return and volatility, the Sharpe, Sortino and Calmar ratios, the maximum drawdown and the CVaR, and, against a benchmark, the excess return, tracking error, information ratio and turnover. The probabilistic and deflated Sharpe ratios ask whether a Sharpe ratio shows skill, or the luck of a search over many trials [67], [68].
- Attribution: the predicted or realised volatility and mean return of a portfolio, split into systematic, idiosyncratic and unexplained parts, by factor, by factor family and by asset, as well as Brinson attribution.
- Forecast evaluation: the information coefficient, coverage, quantile spread, decay and calibration of a return forecast out of sample, with a test of the mean information coefficient that pairs a fixed-bandwidth long-run variance with a wild bootstrap, so the overlap of the forward windows does not inflate its significance. Covariance forecasts are scored against the returns realised after them, step by step through a walk-forward.
- Regret: the log-wealth regret of a strategy against any comparator, with a Newey-West test of equal expected log growth [69]. A comparator fitted in hindsight on each row gives the dynamic regret, and the best sequence of allocations under a budget on its path length gives the regret against the budgeted comparator of [70].

2.15 Design philosophy

PortfolioOptimisers.jl is implemented in Julia [71], using the JuMP.jl mathematical optimisation embedded language [57], and leverages other well-established Julia libraries for much of its functionality. Aside from a strong commitment to FOSS principles, the library is designed atop four pillars:

1. Well-defined type hierarchies. This lets us take advantage of generic fallbacks, and enables fearless extensibility, even at the user-level thanks to multiple dispatch.
2. Strongly typed, immutable structs. This lets the compiler aggressively optimise code, provides a single source of truth, and enables construction-time data validation that will not expire.
3. Compositional design. This lets us build complex objects from simpler ones, enables code reuse, and aids in extensibility.

4. Defensive programming. This lets us catch errors as early as possible, and provides a clear and consistent interface to the user.

With the exception of covariance estimators, which are defined in StatsBase.jl, every type in the library is a subtype of one of three types:

1. `AbstractEstimator`: these contain all the parameters needed to estimate a particular quantity, and are consumed by the functions that perform the estimation. They are the user-level vocabulary, they nest within each other, and they build more complex estimators.
2. `AbstractResult`: many estimators return more than one quantity, and those are returned in a result type. Estimators are configuration; results are data.
3. `AbstractAlgorithm`: these modify the behaviour of estimators via multiple dispatch. They are not used standalone, but as part of an estimator.

The strict adherence to a well-defined type hierarchy lets developers and users alike extend and modify the library's behaviour without modifying the source code. A new risk measure, estimator, or optimiser can live in a user's own package and still compose with everything described here.

3 Availability

PortfolioOptimisers.jl is registered in the Julia General registry, so `Pkg.add("PortfolioOptimisers")` is all that is needed, and it is available under an MIT license. It follows common software development best practices:

1. An extensive and high coverage test suite.
2. Automated documentation builds with a programmatically generated capability catalogue, an introductory user guide, deep examples, and fully documented public and private APIs.
3. Miscellaneous code quality checks.

All as part of a GitHub continuous integration pipeline. The listings in this paper are executed when it is built, so the paper is checked by the same standard as the rest of the project. The library is under active development, the roadmap is public, and contributions are welcome. We hope readers will find it a good place to run their own experiments.

Bibliography

[1] H. Markowitz, "Portfolio selection," *The Journal of Finance*, vol. 7, no. 1, pp. 77–91, 1952.

[2] R. O. Michaud, "The Markowitz optimization enigma: Is 'optimized' optimal?," *Financial Analysts Journal*, vol. 45, no. 1, pp. 31–42, 1989.

[3] M. J. Best and R. R. Grauer, "On the sensitivity of mean-variance-efficient portfolios to changes in asset means: some analytical and computational results," *The Review of Financial Studies*, vol. 4, no. 2, pp. 315–342, 1991.

[4] V. K. Chopra and W. T. Ziemba, "The effect of errors in means, variances, and covariances on optimal portfolio choice," *The Journal of Portfolio Management*, vol. 19, no. 2, pp. 6–11, 1993.

[5] V. DeMiguel, L. Garlappi, and R. Uppal, "Optimal versus naive diversification: How inefficient is the 1/N portfolio strategy?," *The Review of Financial Studies*, vol. 22, no. 5, pp. 1915–1953, 2009.

[6] R. Jagannathan and T. Ma, "Risk reduction in large portfolios: Why imposing the wrong constraints helps," *The Journal of Finance*, vol. 58, no. 4, pp. 1651–1683, 2003.

[7] O. Ledoit and M. Wolf, "A well-conditioned estimator for large-dimensional covariance matrices," *Journal of Multivariate Analysis*, vol. 88, no. 2, pp. 365–411, 2004.

[8] V. DeMiguel, L. Garlappi, F. J. Nogales, and R. Uppal, "A generalized approach to portfolio optimization: Improving performance by constraining portfolio norms," *Management Science*, vol. 55, no. 5, pp. 798–812, 2009.

[9] P. N. Kolm, R. Tütüncü, and F. J. Fabozzi, "60 Years of portfolio optimization: Practical challenges and current trends," *European Journal of Operational Research*, vol. 234, no. 2, pp. 356–371, 2014.

[10] D. Cajas, *ADVANCED PORTFOLIO OPTIMIZATION*. Springer, 2025.

[11] M. M. L. De Prado, *Machine learning for asset managers*. Cambridge University Press, 2020.

[12] P. P. DANIEL, *Portfolio Optimization: Theory and Application*. CAMBRIDGE University Press, 2025.

[13] D. P. Palomar, "Portfolio Optimization." [Online]. Available: https://github.com/dppalomar

[14] D. Cajas, "Riskfolio-Lib (7.3)." [Online]. Available: https://github.com/dcajasn/Riskfolio-Lib

[15] H. Delatte, C. Nicolini, and M. Manzi, "skfolio." [Online]. Available: https://doi.org/10.5281/zenodo.16148630

[16] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[17] S. Diamond and S. Boyd, "CVXPY: A Python-embedded modeling language for convex optimization," *Journal of Machine Learning Research*, vol. 17, no. 83, pp. 1–5, 2016.

[18] A. Agrawal, R. Verschueren, S. Diamond, and S. Boyd, "A rewriting system for convex optimization problems," *Journal of Control and Decision*, vol. 5, no. 1, pp. 42–60, 2018.

[19] A. Musolas, S. T. Smith, and Y. Marzouk, "Geodesically parameterized covariance estimation," *SIAM Journal on Matrix Analysis and Applications*, vol. 42, no. 2, pp. 528–556, 2021, doi: 10.1137/19M1284646.

[20] S. Gerber, H. Markowitz, P. Ernst, Y. Miao, P. Sargen, and others, "The Gerber statistic: A robust co-movement measure for portfolio optimization," *No and Sargen, Paul, The Gerber Statistic: A Robust Co-Movement Measure for Portfolio Optimization (July 4, 2021)*, 2021.

[21] W. Smyth and D. Broby, "An enhanced Gerber statistic for portfolio optimization," *Finance Research Letters*, vol. 49, p. 103229, 2022.

[22] S. Gerber, W. Smyth, H. Markowitz, Y. Miao, P. Ernst, and P. Sargen, "Squeezing financial noise: A novel approach to covariance matrix estimation," *Available at SSRN 4986939*, 2025.

[23] V. A. Marčenko and L. A. Pastur, "Distribution of eigenvalues for some sets of random matrices," *Mathematics of the USSR-Sbornik*, vol. 1, no. 4, p. 457, 1967.

[24] W. Barfuss, G. P. Massara, T. Di Matteo, and T. Aste, "Parsimonious modeling with information filtering networks," *Phys. Rev. E*, vol. 94, no. 6, p. 62306, Dec. 2016, doi: 10.1103/PhysRevE.94.062306.

[25] B. Li and S. C. H. Hoi, "Online portfolio selection: A survey," *ACM Computing Surveys*, vol. 46, no. 3, pp. 1–36, 2014, doi: 10.1145/2512962.

[26] Z. Bar-Joseph, D. K. Gifford, and T. S. Jaakkola, "Fast optimal leaf ordering for hierarchical clustering," *Bioinformatics*, vol. 17, no. suppl_1, pp. S22–S29, 2001, doi: 10.1093/bioinformatics/17.suppl_1.S22.

[27] W.-M. Song, T. Di Matteo, and T. Aste, "Hierarchical information clustering by means of topologically embedded graphs," *PloS one*, vol. 7, no. 3, p. e31929, 2012.

[28] W.-M. Song, T. D. Matteo, and T. Aste, "Nested hierarchies in planar graphs," *Discrete Applied Mathematics*, vol. 159, no. 17, pp. 2135–2146, 2011, doi: https://doi.org/10.1016/j.dam.2011.07.018.

[29] G. P. Massara, T. Di Matteo, and T. Aste, "Network Filtering for Big Data: Triangulated Maximally Filtered Graph," *Journal of Complex Networks*, vol. 5, no. 2, pp. 161–178, Jun. 2016, doi: 10.1093/comnet/cnw015.

[30] F. Black and R. Litterman, "Global portfolio optimization," *Financial Analysts Journal*, vol. 48, no. 5, pp. 28–43, 1992.

[31] A. Meucci, "Fully flexible views: Theory and practice," *Risk*, vol. 21, no. 10, pp. 97–102, 2008.

[32] D. Cajas, "Entropy Pooling with CVaR and EVaR Views," *Available at SSRN 7120258*, 2026.

[33] C. Genest and J. V. Zidek, "Combining probability distributions: A critique and an annotated bibliography," *Statistical Science*, vol. 1, no. 1, pp. 114–135, 1986.

[34] R. C. Grinold and R. N. Kahn, *Active Portfolio Management: A Quantitative Approach for Producing Superior Returns and Controlling Risk*, 2nd ed. New York: McGraw-Hill, 1999.

[35] R. T. Rockafellar and S. Uryasev, "Optimization of conditional value-at-risk," *Journal of Risk*, vol. 2, no. 3, pp. 21–41, 2000.

[36] A. Ahmadi-Javid, "Entropic value-at-risk: A new coherent risk measure," *Journal of Optimization Theory and Applications*, vol. 155, no. 3, pp. 1105–1123, 2012.

[37] P. Mohajerin Esfahani and D. Kuhn, "Data-driven distributionally robust optimization using the Wasserstein metric: performance guarantees and tractable reformulations," *Mathematical Programming*, vol. 171, no. 1, pp. 115–166, 2018.

[38] A. Chekhlov, S. Uryasev, and M. Zabarankin, "Drawdown measure in portfolio optimization," *International Journal of Theoretical and Applied Finance*, vol. 8, no. 1, pp. 13–58, 2005.

[39] D. Cajas, "OWA portfolio optimization: A disciplined convex programming framework," *Available at SSRN 3988927*, 2021.

[40] D. Cajas, "Higher order moment portfolio optimization with L-moments," *Available at SSRN 4393155*, 2023.

[41] D. Cajas, "Efficient Gini Mean Difference and Tail Gini Portfolio Optimization based on P-Norms," *Available at SSRN 4711326*, 2024.

[42] K. E. Batcher, "Sorting networks and their applications," in *Proceedings of the April 30–May 2, 1968, Spring Joint Computer Conference*, 1968, pp. 307–314. doi: 10.1145/1468075.1468121.

[43] R. H. Tütüncü and M. Koenig, "Robust asset allocation," *Annals of Operations Research*, vol. 132, no. 1, pp. 157–187, 2004.

[44] D. Goldfarb and G. Iyengar, "Robust portfolio selection problems," *Mathematics of Operations Research*, vol. 28, no. 1, pp. 1–38, 2003.

[45] R. Zhou and D. P. Palomar, "Understanding the Quintile Portfolio," *IEEE Transactions on Signal Processing*, vol. 68, pp. 4030–4040, 2020, doi: 10.1109/TSP.2020.3006761.

[46] T. M. Cover, "Universal portfolios," *Mathematical Finance*, vol. 1, no. 1, pp. 1–29, 1991.

[47] D. P. Helmbold, R. E. Schapire, Y. Singer, and M. K. Warmuth, "On-line portfolio selection using multiplicative updates," *Mathematical Finance*, vol. 8, no. 4, pp. 325–347, 1998.

[48] A. Agarwal, E. Hazan, S. Kale, and R. E. Schapire, "Algorithms for portfolio management based on the Newton method," in *Proceedings of the 23rd International Conference on Machine Learning*, 2006, pp. 9–16.

[49] B. Li, P. Zhao, S. C. H. Hoi, and V. Gopalkrishnan, "PAMR: Passive aggressive mean reversion strategy for portfolio selection," *Machine Learning*, vol. 87, no. 2, pp. 221–258, 2012.

[50] B. Li, S. C. H. Hoi, D. Sahoo, and Z.-Y. Liu, "Moving average reversion strategy for on-line portfolio selection," *Artificial Intelligence*, vol. 222, pp. 104–123, 2015.

[51] D. Huang, J. Zhou, B. Li, S. C. H. Hoi, and S. Zhou, "Robust median reversion strategy for online portfolio selection," *IEEE Transactions on Knowledge and Data Engineering*, vol. 28, no. 9, pp. 2480–2493, 2016.

[52] B. Li, S. C. H. Hoi, P. Zhao, and V. Gopalkrishnan, "Confidence weighted mean reversion strategy for online portfolio selection," *ACM Transactions on Knowledge Discovery from Data*, vol. 7, no. 1, p. 4:1–4:38, 2013.

[53] A. Borodin, R. El-Yaniv, and V. Gogan, "Can we learn to beat the best stock," *Journal of Artificial Intelligence Research*, vol. 21, pp. 579–594, 2004.

[54] B. Li, J. Wang, D. Huang, and S. C. H. Hoi, "Transaction cost optimization for online portfolio selection," *Quantitative Finance*, vol. 18, no. 8, pp. 1411–1424, 2018.

[55] L. Györfi, G. Lugosi, and F. Udina, "Nonparametric kernel-based sequential investment strategies," *Mathematical Finance*, vol. 16, no. 2, pp. 337–357, 2006, doi: 10.1111/j.1467-9965.2006.00274.x.

[56] L. Zhang, S. Lu, and Z.-H. Zhou, "Adaptive online learning in dynamic environments," in *Advances in Neural Information Processing Systems 31*, 2018, pp. 1323–1333.

[57] M. Lubin, O. Dowson, J. Dias Garcia, J. Huchette, B. Legat, and J. P. Vielma, "JuMP 1.0: Recent improvements to a modeling language for mathematical optimization," *Mathematical Programming Computation*, vol. 15, pp. 581–589, 2023, doi: 10.1007/s12532-023-00239-3.

[58] P. J. Goulart and Y. Chen, "Clarabel: An interior-point solver for conic programs with quadratic objectives." [Online]. Available: https://arxiv.org/abs/2405.12762

[59] S. Maillard, T. Roncalli, and J. Teiletche, "The properties of equally weighted risk contribution portfolios," *The Journal of Portfolio Management*, vol. 36, no. 4, pp. 60–70, 2010.

[60] M. L. De Prado, "Building diversified portfolios that outperform out of sample," *The Journal of Portfolio Management*, vol. 42, no. 4, pp. 59–69, 2016.

[61] T. Raffinot, "Hierarchical clustering-based asset allocation," *The Journal of Portfolio Management*, vol. 44, no. 2, pp. 89–99, 2017.

[62] P. Cotton, "Schur Complementary Allocation: A Unification of Hierarchical Risk Parity and Minimum Variance Portfolios." [Online]. Available: https://arxiv.org/abs/2411.05807

[63] D. H. Wolpert, "Stacked generalization," *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.

[64] W. Shen and J. Wang, "Portfolio selection via subset resampling," in *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence*, 2017, pp. 1517–1523.

[65] M. M. L. De Prado, *Advances in financial machine learning*. John Wiley & Sons, 2018.

[66] M. Besançon *et al.*, "Distributions.jl: Definition and Modeling of Probability Distributions in the JuliaStats Ecosystem," *Journal of Statistical Software*, vol. 98, no. 16, pp. 1–30, 2021, doi: 10.18637/jss.v098.i16.

[67] D. H. Bailey and M. Lopez de Prado, "The Sharpe ratio efficient frontier," *Journal of Risk*, vol. 15, no. 2, pp. 3–44, 2012.

[68] D. H. Bailey and M. Lopez de Prado, "The deflated Sharpe ratio: correcting for selection bias, backtest overfitting, and non-normality," *The Journal of Portfolio Management*, vol. 40, no. 5, pp. 94–107, 2014.

[69] W. K. Newey and K. D. West, "A simple, positive semi-definite, heteroskedasticity and autocorrelation consistent covariance matrix," *Econometrica*, vol. 55, no. 3, pp. 703–708, 1987, doi: 10.2307/1913610.

[70] M. Zinkevich, "Online convex programming and generalized infinitesimal gradient ascent," in *Proceedings of the 20th International Conference on Machine Learning*, 2003, pp. 928–936.

[71] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, "Julia: A fresh approach to numerical computing," *SIAM Review*, vol. 59, no. 1, pp. 65–98, 2017, doi: 10.1137/141000671.